UNIT – V

E-Mail Security & IP Security (IPSec)

PART A: E-Mail Security

- Pretty Good Privacy (PGP)
  - Services, Operation, Keys
  - PGP Message Format
- S/MIME
  - MIME, S/MIME Services
  - Content Types, Certificate

PART B: IP Security (IPSec)

- IPSec Overview & Architecture
- Authentication Header (AH)
- Encapsulating Security Payload (ESP)
- Combining Security Associations
- Internet Key Exchange (IKE)
  - IKEv1 & IKEv2

PART A: E-MAIL SECURITY

1. Pretty Good Privacy (PGP)

1.1 Introduction

Pretty Good Privacy (PGP) is a data-encryption and decryption program that provides cryptographic privacy and authentication for data communication. It was created by Phil Zimmermann in 1991 and has become the de-facto standard for email security worldwide. PGP uses a combination of symmetric-key cryptography and public-key cryptography to provide a practical, widely deployable security solution.

1.2 Why PGP?

- **Available free:** Original implementation freely available worldwide.
- **Platform independent:** Runs on a wide variety of platforms (Unix, Windows, macOS, mobile).
- **Based on well-studied algorithms:** Uses RSA/DSS, IDEA/3DES/AES, SHA — all well-analysed.
- **Wide applicability:** Applicable to email, file encryption, and secure storage.
- **Not government-controlled:** Developed independently; trusted by individuals and organisations globally.

1.3 PGP Services

PGP provides five categories of services:

Service	Algorithm Used	Description
Authentication	DSS/RSA + SHA-1/SHA-256	Digital signature; sender signs message hash with private key
Confidentiality	CAST-128 / 3DES / IDEA / AES	Symmetric encryption of message; session key encrypted with recipient's public key
Compression	ZIP	Message compressed before encryption to save bandwidth and increase security
Email Compatibility	Radix-64 (Base64)	Binary ciphertext converted to printable ASCII for email transmission
Segmentation	Fragmentation	Long messages split to respect email size limits; reassembled at receiver

1.4 PGP Operation – Step-by-Step

Sending a Signed and Encrypted Message

Step-by-Step PGP Send Operation

1. **Step 1 – Create Message:** Sender (Alice) creates plaintext message M.
2. **Step 2 – Hash:** SHA-256 hash H(M) is computed.
3. **Step 3 – Sign:** Alice signs hash using her RSA/DSS private key (KRa): S = E(KRa, H(M)).
4. **Step 4 – Concatenate:** Signature S is prepended to message M: [S || M].
5. **Step 5 – Compress:** ZIP compression applied to [S || M].
6. **Step 6 – Session Key:** Alice generates a one-time random symmetric session key Ks.
7. **Step 7 – Encrypt Message:** Compressed data encrypted with Ks using AES/CAST-128: E(Ks, ZIP[S||M]).
8. **Step 8 – Encrypt Session Key:** Ks encrypted with Bob's public key KUb: E(KUb, Ks).
9. **Step 9 – Combine:** [E(KUb,Ks) || E(Ks,ZIP[S||M])] forms the PGP message.
10. **Step 10 – Radix-64:** Binary output converted to ASCII using Base64 for email transmission.

Receiving and Verifying a PGP Message

Step-by-Step PGP Receive Operation

11. **Step 1 – Decode:** Bob decodes Radix-64 to get binary PGP message.
12. **Step 2 – Decrypt Session Key:** Bob decrypts session key: Ks = D(KRb, E(KUb,Ks)).
13. **Step 3 – Decrypt Message:** Using Ks, Bob decrypts: ZIP[S||M] = D(Ks, ciphertext).
14. **Step 4 – Decompress:** ZIP decompression recovers [S || M].
15. **Step 5 – Separate:** Signature S and message M are separated.
16. **Step 6 – Verify:** Bob computes H(M) and decrypts S using Alice's public key KUa: H' = D(KUa, S).
17. **Step 7 – Compare:** If H(M) = H', signature is valid — message is authentic and unaltered.

1.5 PGP Message Format

Component	Description / Content
Message Component	Filename, timestamp, literal data (the actual message body)
Signature Component	Timestamp, key ID of signer, hash algorithm ID, leading octets of hash, signature (RSA/DSS)

Session Key Component	Key ID of recipient's public key, encrypted session key (asymmetrically encrypted)

1.6 PGP Key Rings

PGP maintains two key ring data structures on each node:

Key Ring	Contents
Private Key Ring	Stores the user's own private/public key pairs; private key stored encrypted (passphrase-protected)
Public Key Ring	Stores public keys of other users; indexed by key ID; used to look up keys for encryption and verification

1.7 PGP Trust Model – Web of Trust

Unlike certificate authorities used in PKI, PGP uses a decentralised trust model called the Web of Trust. There is no single trusted authority; instead, trust is built through personal endorsements.

Trust Level	Meaning
Unknown	Nothing known about the key owner's trust in signing others' keys
Untrusted	Owner is known to be unreliable in validating keys before signing
Marginal	Owner is considered partially trustworthy; requires 2–3 marginal trusts to validate a key
Full	Owner is fully trusted to validate and sign others' public keys
Ultimate	User's own key; implicitly trusted

A public key is considered valid if it is directly signed by a fully-trusted introducer, or signed by 2 or more marginally-trusted introducers.

1.8 PGP Session Key Generation

PGP generates session keys using a PRNG seeded with random events — keystrokes, mouse movements, elapsed time between keystrokes. The key is used for exactly one message and then discarded, providing perfect forward secrecy at the message level.

1.9 Radix-64 (ASCII Armoring)

Email systems historically only support 7-bit ASCII text. Encrypted binary data must be converted for reliable email delivery. PGP uses Radix-64 encoding (Base64) which converts every 3 bytes of binary data into 4 printable ASCII characters.

- **Expansion factor:** 33% size increase (3 bytes → 4 characters).
- **CRC-24:** PGP also appends a 3-byte CRC-24 checksum to detect transmission errors.
- **Header lines:** PGP ASCII-armored messages are delimited by '-----BEGIN PGP MESSAGE-----' and '-----END PGP MESSAGE-----' lines.

2. S/MIME – Secure/Multipurpose Internet Mail Extension

2.1 Background: MIME

MIME (Multipurpose Internet Mail Extensions, RFC 2045–2049) extends basic email to support:

- Non-ASCII character sets (international languages)
- Non-text attachments (images, audio, video, application data)
- Multi-part message bodies
- Header information in non-ASCII character sets

MIME Header Field	Purpose
MIME-Version	Declares MIME version (must be 1.0)
Content-Type	Specifies media type and subtype of body (e.g., text/plain, image/jpeg)
Content-Transfer-Encoding	Defines encoding (7bit, 8bit, base64, quoted-printable)
Content-ID	Unique identifier for MIME entity
Content-Description	Text description of body (useful for non-text entities)

2.2 What is S/MIME?

S/MIME (Secure/MIME) is a standard for public-key encryption and signing of MIME data. It is defined in RFC 5751 and is built into virtually all modern email clients (Outlook, Apple Mail, Thunderbird). Unlike PGP's Web of Trust, S/MIME uses a hierarchical X.509 certificate infrastructure (PKI).

2.3 S/MIME Services

Service	Function	Algorithm
Enveloped Data	Encrypt message content + session key for recipients	AES-128/256 (content); RSA or ECDH (key)
Signed Data	Digitally sign message; optional base64 encoding	RSA/DSA/ECDSA with SHA-256/384/512

Clear-Signed Data	Sign without encrypting; non-S/MIME clients can read body	Detached signature; body readable as plain text
Signed & Enveloped	Both signing and encryption combined	Nested CMS structures
Compressed Data	Compress using zlib before other operations	zlib (RFC 3274)

2.4 S/MIME Content Types

MIME Type	Purpose
application/pkcs7-mime (smime-type=enveloped-data)	Encrypted message (CMS EnvelopedData)
application/pkcs7-mime (smime-type=signed-data)	Opaque signed message (signer info + content combined)
multipart/signed	Clear-signed message (body + detached signature as separate parts)
application/pkcs7-signature	Detached signature (used with multipart/signed)
application/pkcs7-mime (smime-type=certs-only)	Certificate message — carries certificate chain only
application/pkcs7-mime (smime-type=compressed-data)	Compressed message

2.5 S/MIME Certificate

S/MIME requires an X.509 v3 certificate to associate a user's identity with their public key. The certificate is issued by a trusted Certificate Authority (CA) and must include:

- **Subject:** User's distinguished name and email address
- **Public Key:** RSA (2048+ bits) or ECDSA (P-256 or higher)
- **Key Usage Extensions:** Digital Signature and/or Key Encipherment flags set
- **CA Signature:** Certificate signed by trusted CA chain

2.6 S/MIME CMS (Cryptographic Message Syntax)

S/MIME uses CMS (RFC 5652) — previously known as PKCS #7 — as its underlying message format. CMS defines ASN.1 structures for signed, encrypted, and other secured data.

CMS Content Type	Description
Data	Raw octet string — the actual message body

SignedData	Arbitrary data + one or more digital signatures
EnvelopedData	Encrypted content + encrypted session keys for one or more recipients
DigestedData	Arbitrary data + its message digest (for integrity without authentication)
EncryptedData	Encrypted content without key info (for symmetric-key scenarios)
AuthenticatedData	Data + MAC (authentication without non-repudiation)

2.7 PGP vs S/MIME Comparison

Feature	PGP	S/MIME
Trust Model	Web of Trust (decentralised)	Hierarchical PKI / CA (centralised)
Certificate	PGP keys / self-signed	X.509 v3 certificates
Standard Body	OpenPGP (RFC 4880)	IETF (RFC 5751)
Key Management	User-managed key rings	CA-managed; directory-based
Email Client Support	Plugin required (Enigmail, GPG Suite)	Built into Outlook, Apple Mail, etc.
Enterprise Adoption	Common in technical communities	Dominant in corporate environments
Key Revocation	Key revocation certificates	CRL / OCSP via CA
Compression	ZIP before encryption	zlib (optional per RFC 3274)

PART B: IP SECURITY (IPSec)

3. IP Security – Overview

3.1 Introduction

IPSec (Internet Protocol Security) is a suite of protocols developed by the IETF to secure IP communications by authenticating and/or encrypting each IP packet in a communication session. It operates at the Network Layer (Layer 3) and is transparent to applications — no changes to application software are required.

3.2 Why IPSec?

The original IP protocol was designed for an open, trusting academic network and has no built-in security. As the Internet grew into a critical infrastructure, security gaps became serious threats:

- **No source authentication:** IP source address can be forged (IP spoofing).
- **No data integrity:** IP packets can be modified in transit without detection.
- **No confidentiality:** IP packet payload is transmitted in cleartext.
- **No replay protection:** Captured packets can be re-transmitted to achieve unauthorized effects.

IPSec addresses all these issues by providing:

- **Authentication:** Verifies that packets were sent by the claimed source.
- **Integrity:** Detects any modification to packets in transit.
- **Confidentiality:** Encrypts packet payload so only intended recipients can read it.
- **Anti-replay:** Sequence numbers prevent captured packets from being replayed.
- **Key management:** Automated key establishment and management via IKE.

3.3 IPSec Applications

Application	Description
VPN (Virtual Private Network)	Secure tunnel over untrusted Internet between two sites or a remote user and a corporate network
Secure remote access	Road warriors connecting securely to corporate resources from hotels, airports
B2B connectivity	Secure partner-to-partner communication across the Internet
Intranet protection	Secure sensitive departments even on an internal network
IPv6 native	IPSec is mandatory in IPv6; optional but widely deployed in IPv4

3.4 IPSec Modes of Operation

Mode	Applied To	Use Case
Transport Mode	IP payload only; original IP header retained	End-to-end host-to-host security; no tunnel
Tunnel Mode	Entire original IP packet (header + payload) encapsulated inside new IP packet	Gateway-to-gateway VPN; remote access VPN

Transport Mode vs Tunnel Mode Packet Structure

Transport Mode:

[Original IP Header | AH / ESP Header | IP Payload]

Tunnel Mode:

[New IP Header | AH / ESP Header | Original IP Header | IP Payload]

In Tunnel Mode, the original IP header is hidden — intermediate routers see only the outer IP header.

4. IP Security Architecture

4.1 IPSec Document Structure

IPSec is not a single protocol — it is a framework of several documents and components:

RFC / Component	Purpose
RFC 4301 – Security Architecture	Overall IPSec architecture and concepts
RFC 4302 – Authentication Header (AH)	Authentication + integrity protocol
RFC 4303 – Encapsulating Security Payload (ESP)	Confidentiality + authentication + integrity protocol
RFC 4306/7296 – IKE / IKEv2	Key management and security association negotiation
RFC 4835 – Cryptographic Algorithms	Mandatory and recommended cipher suites
RFC 4307 – IKEv2 Algorithms	Cipher suites for IKEv2

4.2 Security Associations (SA)

A Security Association (SA) is the fundamental concept of IPSec. It is a one-way logical connection between a sender and receiver that defines the security parameters for a communication. All traffic within an SA is treated the same way.

An SA is uniquely identified by the triple:

- **Security Parameter Index (SPI):** A 32-bit value that identifies the SA to the receiver.
- **Destination IP Address:** The IP address of the SA endpoint (unicast, multicast, or broadcast).
- **Security Protocol:** Identifies whether AH or ESP is used for this SA.

Each SA stores the following parameters in the Security Association Database (SAD):

- **Sequence Number Counter:** 32-bit value for generating the Sequence Number field.
- **Sequence Counter Overflow:** Flag whether overflow should cause an audit event and prevent further packets.
- **Anti-Replay Window:** 64-bit window for detecting replayed packets.
- **AH Information:** Authentication algorithm, keys, key lifetimes.
- **ESP Information:** Encryption and authentication algorithms, keys, IV, key lifetimes.
- **SA Lifetime:** Time or byte count after which the SA must be replaced or terminated.
- **IPSec Protocol Mode:** Tunnel or Transport mode.
- **MTU Path:** Maximum Transmission Unit along path.

4.3 SA Bundle and Iterated SA

A single SA applies only one security protocol (AH or ESP). For more complex requirements, multiple SAs can be chained:

- **SA Bundle:** A sequence of SAs through which traffic must pass. Each SA in the bundle is applied in sequence.
- **Iterated tunneling:** Multiple nested tunnels; used for complex multi-hop VPN scenarios.

5. Authentication Header (AH)

5.1 Purpose

The IP Authentication Header (AH), defined in RFC 4302, provides:

- **Data origin authentication:** Verifies the identity of the packet sender.
- **Connectionless integrity:** Ensures the packet has not been modified in transit.
- **Anti-replay protection:** Optional; rejects duplicate packets using sequence numbers.

> **Note:** AH does NOT provide confidentiality (encryption). If confidentiality is required, use ESP. AH authenticates the entire IP packet including the (immutable parts of the) outer IP header.

5.2 AH Header Format

Field	Size / Description
Next Header	8 bits — identifies the type of the next payload (e.g., TCP=6, UDP=17, ESP=50)
Payload Length	8 bits — length of AH in 32-bit words minus 2 (e.g., value 4 = 24 bytes total AH)
Reserved	16 bits — set to zero
Security Parameters Index (SPI)	32 bits — identifies the Security Association for this packet
Sequence Number	32 bits — monotonically increasing counter; starts at 1; enables anti-replay
Integrity Check Value (ICV)	Variable length (32-bit aligned) — HMAC or AES-XCBC-MAC over protected fields

5.3 AH in Transport Mode

In Transport Mode, AH is inserted between the IP header and the upper-layer protocol (TCP/UDP):

AH Transport Mode Packet Structure

IPv4: [IP Header | AH Header | Upper Layer (TCP/UDP/...) | Data]

IPv6: [IPv6 Header | Hop-by-Hop | Dest Opts | AH Header | Routing | Dest Opts | TCP/UDP | Data]

Authenticated fields: Immutable IPv4 header fields + AH header + entire upper layer payload.

Mutable fields (TTL, ToS, Checksum) are zeroed before ICV computation.

5.4 AH in Tunnel Mode

In Tunnel Mode, the entire original packet becomes the payload of a new IP packet. AH is inserted between the new outer IP header and the original IP packet:

AH Tunnel Mode Packet Structure

[New IP Header | AH Header | Original IP Header | Original Payload]

Authenticated fields: Immutable fields of new outer IP header + AH + entire original packet (original IP header + payload).

The original inner IP header is fully protected — this is a key advantage of Tunnel Mode.

6. Encapsulating Security Payload (ESP)

6.1 Purpose

ESP (Encapsulating Security Payload), defined in RFC 4303, provides a richer set of security services than AH:

- **Confidentiality (encryption):** Encrypts the payload to hide contents from eavesdroppers.
- **Data origin authentication:** Verifies the identity of the sender (optional but recommended).
- **Connectionless integrity:** Verifies data has not been tampered with in transit.
- **Anti-replay protection:** Detects and rejects replayed packets using sequence numbers.
- **Limited traffic flow confidentiality:** In tunnel mode, hides original source/destination IP addresses.

6.2 ESP Header and Trailer Format

Field	Size / Description
Security Parameters Index (SPI)	32 bits — identifies the SA (part of ESP header)
Sequence Number	32 bits — anti-replay counter, starts at 1 (part of ESP header)
Payload Data	Variable — the encrypted upper-layer data or original IP packet (tunnel mode)
Padding	0–255 bytes — aligns plaintext to block cipher boundary
Pad Length	8 bits — number of padding bytes (0 if none)
Next Header	8 bits — type of data in Payload Data field (e.g., TCP=6, IP-in-IP=4)
Integrity Check Value (ICV)	Variable (optional) — covers SPI + Seq No + Payload + Padding + Pad Length + Next Header

6.3 ESP Packet Processing

Encryption Scope

ESP encrypts: Payload Data, Padding, Pad Length, and Next Header. The SPI and Sequence Number are NOT encrypted (they are in the clear so the receiver can identify the SA and check for replay).

Authentication Scope (when ICV present)

The ICV(Integrity Check Value) covers: SPI + Sequence Number + Payload Data + Padding + Pad Length + Next Header. This authenticated-encryption protects against both eavesdropping and modification.

6.4 ESP in Transport Mode

ESP Transport Mode Structure

[IP Header | ESP Header | TCP/UDP Header | Data | ESP Trailer | ESP ICV]

Encrypted: TCP/UDP Header + Data + ESP Trailer

Authenticated: ESP Header + Encrypted fields (SPI onward to ESP Trailer)

NOT authenticated: Original IP Header (use AH+ESP bundle to authenticate IP header)

6.5 ESP in Tunnel Mode

ESP Tunnel Mode Structure

[New IP Header | ESP Header | Original IP Header | Original Payload | ESP Trailer | ESP ICV]

Encrypted: Original IP Header + Original Payload + ESP Trailer

Authenticated: ESP Header + all encrypted fields

NOT visible to intermediate routers: Original source/destination IP addresses — traffic flow confidentiality

6.6 ESP vs AH Comparison

Feature	AH	ESP
Confidentiality	No	Yes (mandatory component)
Data Origin Authentication	Yes	Yes (optional but recommended)
Connectionless Integrity	Yes	Yes (when auth enabled)
Anti-Replay	Yes (optional)	Yes (optional)
IP Header Authentication	Yes (outer + inner in tunnel)	No (outer IP header not covered)
Traffic Flow Confidentiality	No	Yes (tunnel mode hides inner IP header)
IP Protocol Number	51	50
NAT Traversal	Breaks NAT (authenticates outer IP)	Compatible with NAT-T (UDP 4500)
Best Use Case	Integrity without confidentiality	Confidentiality + integrity (most common)

6.7 ESP Encryption Algorithms

7. Combining Security Associations

7.1 Introduction

A single SA can implement either AH or ESP, but not both. To meet complex security requirements, multiple SAs can be combined into a bundle. An SA bundle is a sequence of SAs through which traffic must pass. There are two ways to combine SAs:

- **Transport Adjacency:** Two SAs in transport mode applied to the same IP packet (no tunneling).
- **Iterated Tunneling:** Multiple levels of tunneling using AH and/or ESP.

7.2 Transport Adjacency

Applying two SAs in transport mode to the same packet. The only valid combination is ESP (inner) + AH (outer):

Transport Adjacency: ESP + AH

1. Inner SA: Apply ESP in transport mode → encrypts TCP/UDP payload
   [IP Header | ESP Header | TCP | Data | ESP Trailer | ESP ICV]

2. Outer SA: Apply AH in transport mode → authenticates entire packet including IP header
   [IP Header | AH Header | ESP Header | TCP | Data | ESP Trailer | ESP ICV | AH ICV]

Result: Authentication covers entire packet; encryption covers payload. Most comprehensive transport mode combination.

7.3 Iterated Tunneling

Multiple nested tunnels providing different security services at each level. Common scenarios:

Scenario 1: End-to-End + Tunnel (Host to Host via Gateway)

Host-to-Host SA + Gateway Tunnel SA

Step 1: Host A applies ESP in transport mode (end-to-end SA between hosts):
[IP(A→B) | ESP | TCP | Data | ESP-Trailer | ICV]

Step 2: Gateway GW1 applies ESP/AH in tunnel mode (gateway SA):
[IP(GW1→GW2) | ESP | IP(A→B) | ESP | TCP | Data | ESP-Trailer | ICV | ESP-Trailer | ICV]

Step 3: GW2 decapsulates outer tunnel; B decapsulates inner ESP.

Result: End-to-end privacy (host-level SA) + VPN tunnel protection.

Scenario 2: Double Tunnel

Two Nested Tunnels (Host → GW1 → GW2 → Host)

Inner Tunnel SA (Host A to Host B in tunnel mode):

[IP(GW1→GW2) | IPSec | IP(A→B) | TCP | Data]

Outer Tunnel SA (between Gateways GW1 → GW2):

[IP(Internet) | IPSec | IP(GW1→GW2) | IPSec | IP(A→B) | TCP | Data]

Used in multi-hop VPN scenarios where security must be applied at multiple administrative boundaries.

7.4 Basic Combinations Supported by IPSec

Combination	Scenario	Notes
AH transport	Host-to-host integrity	Authentication only; no encryption; NAT-unfriendly
ESP transport	Host-to-host privacy	Encryption + optional auth; most common host mode
ESP transport + AH transport	Host-to-host: privacy + full auth	AH covers IP header; ESP encrypts payload; NAT-unfriendly
AH tunnel	Gateway-to-gateway integrity	Rare; no confidentiality
ESP tunnel	Gateway-to-gateway VPN	Most common VPN deployment (site-to-site, remote access)
ESP tunnel + AH transport	End-to-end + VPN tunnel	Complex; deep security requirement

8. Internet Key Exchange (IKE)

8.1 Introduction

Internet Key Exchange (IKE) is the protocol used to set up Security Associations in IPSec. IKE performs mutual authentication between communicating parties and establishes shared session keys for AH and ESP. It replaces the need for manually configured keys, making large-scale IPSec deployment practical.

Version	RFC	Notes

IKEv1	RFC 2409	Original; complex; two phases; still deployed
IKEv2	RFC 7296 (2014)	Simplified; more efficient; mandatory MOBIKE; preferred

8.2 IKE Building Blocks

- **ISAKMP (Internet Security Association and Key Management Protocol):** Framework defining message formats, state machines, and procedures for SA establishment — RFC 2408.
- **OAKLEY Key Determination Protocol:** Defines a series of key exchanges using Diffie-Hellman — RFC 2412.
- **SKEME:** Key exchange mechanism providing anonymity, repudiation, and quick key refreshment.

IKEv1 combines elements of all three. IKEv2 is a clean redesign.

8.3 IKEv1 – Two Phases

Phase 1 – IKE SA (ISAKMP SA)

Establishes a secure, authenticated channel (the IKE SA) that protects all subsequent IKE communications. Phase 1 can operate in two modes:

Phase 1 Mode	Description
Main Mode (6 messages)	3 rounds of 2 messages; protects identity of both parties; slower but more secure
Aggressive Mode (3 messages)	Faster; exposes identities in cleartext; vulnerable to offline dictionary attacks — avoid

Phase 2 – IPSec SA (Quick Mode)

Uses the IKE SA from Phase 1 to negotiate one or more IPSec SAs (AH/ESP parameters). All messages are encrypted and authenticated using Phase 1 keys.

8.5 IKE Authentication Methods

Method	Description
Pre-Shared Key (PSK)	Simple shared secret; scalability problem for large deployments; common in site-to-site VPN
RSA Digital Signature	Sign AUTH payload with RSA private key; requires certificates or raw public keys
DSS Digital Signature	Sign with DSA; less common than RSA

ECDSA Digital Signature	Elliptic curve signatures; smaller keys; preferred in modern deployments
EAP (IKEv2 only)	Extensible Authentication Protocol; enables username/password, OTP, SIM-based auth via RADIUS

8.6 Perfect Forward Secrecy (PFS)

PFS ensures that session keys are not compromised even if the long-term private key is later compromised. With PFS, a new Diffie-Hellman exchange is performed for each new IPSec SA — the session key is not derivable from the long-term key alone.

- **Without PFS:** Compromise of long-term key + recording of past traffic → decrypt all past sessions.
- **With PFS:** Each session uses a fresh DH exchange → compromise of long-term key reveals nothing about past sessions.

QUICK REVISION SUMMARY

Complete Topic Summary

Topic	Key Concept	Critical Detail / Formula
PGP	Hybrid encryption + digital signature for email	Signs with sender's private key; encrypts with recipient's public key; uses ZIP + Radix-64
PGP Session Key	Random one-time symmetric key	Ks generated per message; E(KUb,Ks) sent with message
PGP Web of Trust	Decentralised trust model	Full / Marginal / Untrusted trust levels; no central CA
PGP Radix-64	Binary → ASCII email compatibility	Base64 + CRC-24; 33% size overhead
S/MIME	PKI-based email security	Uses X.509 certificates + CMS (PKCS#7) message format
S/MIME Services	Sign, Encrypt, Sign+Encrypt, Compress	Content types: pkcs7-mime, multipart/signed
IPSec	Network-layer security suite	Transparent to applications; Transport and Tunnel modes
Security Association	One-way IPSec relationship	Identified by (SPI, Destination IP, Protocol)
AH (Protocol 51)	Authentication + integrity only	Covers IP header; breaks NAT; no encryption

ESP (Protocol 50)	Encryption + optional auth	Does not cover outer IP header; NAT-T compatible
AH Transport	Protects host-to-host	Original IP header + payload authenticated
ESP Tunnel	Most common VPN mode	Entire original packet encrypted; traffic flow confidentiality
IKEv1 Phase 1	Establish IKE SA	Main Mode (6 msgs) or Aggressive Mode (3 msgs)
IKEv1 Phase 2	Establish IPSec SA	Quick Mode (3 msgs); uses IKE SA for protection
IKEv2	Streamlined key exchange	IKE_SA_INIT + IKE_AUTH = 4 messages total
PFS	Perfect Forward Secrecy	New DH per session; past sessions safe if key compromised

Key Acronyms

Acronym	Full Form	Context
PGP	Pretty Good Privacy	Email encryption; Phil Zimmermann 1991
S/MIME	Secure/Multipurpose Internet Mail Extensions	PKI-based email security
CMS	Cryptographic Message Syntax	Underlying format for S/MIME (PKCS#7/RFC 5652)
IPSec	IP Security	Network-layer security suite
SA	Security Association	One-way IPSec relationship
SAD	Security Association Database	Stores active SAs and their parameters
SPD	Security Policy Database	Defines which traffic gets IPSec treatment
SPI	Security Parameters Index	32-bit SA identifier in AH/ESP header
AH	Authentication Header	IPSec authentication-only protocol
ESP	Encapsulating Security Payload	IPSec encryption+auth protocol
IKE	Internet Key Exchange	Automated SA negotiation and key management
ISAKMP	Internet SA and Key Management Protocol	IKE framework standard

PFS	Perfect Forward Secrecy	Fresh DH per session; protects past sessions

Exam Tips – Must Remember

- **PGP:** Signs FIRST (with sender's private key), then encrypts (with recipient's public key). ORDER MATTERS.
- **PGP compression:** Applied AFTER signing, BEFORE encryption — improves compression ratio and security.
- **AH vs ESP:** AH = authentication only (no encryption); ESP = encryption + optional auth. AH breaks NAT; ESP doesn't.
- **SA is one-way:** Two SAs needed for full duplex communication (one for each direction).
- **Transport mode:** Host-to-host; original IP header preserved. Tunnel mode: gateway-to-gateway; entire packet encapsulated.
- **IKEv2 vs IKEv1:** IKEv2 = simpler (4 msgs vs 9), built-in EAP, MOBIKE, better DoS protection.
- **S/MIME uses X.509 certs; PGP uses Web of Trust — key exam distinction.**